20. Docker 빌드 및 배포 과정 분석

개요

Omni 네트워크는 GoReleaser와 Docker를 활용한 자동화된 빌드 및 배포 시스템을 구축하고 있습니다. 이 문서는 Makefile, GoReleaser 설정, 그리고 각 컴포넌트의 Docker 컨테이너 빌드 과정을 상세히 분석합니다.

빌드 시스템 아키텍처

전체 빌드 플로우

```
┌─────────────────┐  ┌─────────────────┐  ┌─────────────────┐
| Source Code |──▶| GoReleaser |──▶| Docker Images |
| | | | | |
| • Go 바이너리 | | • 크로스 컴파일 | | • 멀티 아키텍처 |
| • Dockerfile | | • 아키텍처 매트릭스| | • 태그 관리 |
| • 설정 파일 | | • 바이너리 생성 | | • 레지스트리 푸시|
└─────────────────┘  └─────────────────┘  └─────────────────┘
```

빌드 타겟 분류

외부 사용자 대상 (External Facing):

- `halo` : 합의 클라이언트

- `omni` : CLI 도구

내부 서비스 대상 (Internal Facing):

- `relayer` : 크로스체인 릴레이어
- `monitor` : 모니터링 서비스
- `solver` : SolverNet 구현

Makefile 분석

Docker 빌드 타겟

```
.PHONY: build-docker
build-docker: ensure-go-releaser ## Builds the docker images using local arch.
@git config --local core.abbrev 7
@sed "s/amd64/$(ARCH)/g" .goreleaser-snapshot.yaml > .goreleaser-local.yaml
@FOUNDRY_VERSION=$(FOUNDRY_VERSION) \
GORELEASER_CURRENT_TAG=$(./scripts/go_tag.sh 0) \
GORELEASER_PREVIOUS_TAG=$(./scripts/go_tag.sh 1) \
goreleaser release -f .goreleaser-local.yaml --snapshot --clean --skip=archive
@rm .goreleaser-local.yaml
@scripts/halovisor/build.sh
```

주요 기능:

1. **아키텍처 감지**: `ARCH := $(shell arch | sed 's/x86_64/amd64/')`
2. **동적 설정 생성**: 로컬 아키텍처에 맞는 GoReleaser 설정 생성

3. **버전 태그 관리**: `go_tag.sh` 스크립트로 현재/이전 태그 추출
4. **Foundry 버전 주입**: 스마트 컨트랙트 도구 버전 관리
5. **Halovisor 빌드**: 업그레이드 가능한 컨테이너 생성

개발 환경 타겟

```
.PHONY: devnet-deploy
devnet-deploy: ## Deploys devnet (MANIFEST=devnet1 by default)
@echo "Creating a docker-compose devnet in ./e2e/run/$(if $(MANIFEST),$(MANIFEST
@go run github.com/omni-network/omni/e2e -f e2e/manifests/$(if $(MANIFEST),$(MAN

.PHONY: devnet-clean
devnet-clean: ## Deletes devnet containers (MANIFEST=devnet1 by default)
@echo "Stopping the devnet in ./e2e/run/$(if $(MANIFEST),$(MANIFEST),devnet1)"
@go run github.com/omni-network/omni/e2e -f e2e/manifests/$(if $(MANIFEST),$(MAN
```

GoReleaser 설정 분석

공식 릴리스 설정 (.goreleaser-official.yaml)

빌드 매트릭스

외부 사용자 대상 바이너리:

```
builds:
- id: halo
```

```
main: ./halo
binary: halo
env: [CGO_ENABLED=0]
goos: [linux, darwin] # Linux, macOS 지원
goarch: [amd64, arm64] # Intel, ARM 아키텍처 지원
ldflags:
- -s -w -X github.com/omni-network/omni/lib/buildinfo.version={{.Tag}}
```

내부 서비스 바이너리:

```
- id: relayer
main: ./relayer
binary: relayer
env: [CGO_ENABLED=0]
goos: [linux] # Linux만 지원
goarch: [amd64] # AMD64만 지원
```

Docker 이미지 빌드

멀티 아키텍처 이미지:

```
dockers:
- id: halo-amd64
ids: [halo]
goos: linux
```

```
goarch: amd64
use: buildx  # Docker Buildx 사용
build_flag_templates:
- "--platform=linux/amd64"
dockerfile: ./halo/Dockerfile
image_templates:
- omniops/halo:{{.Tag}}-amd64
```

Docker 매니페스트:

```
docker_manifests:
- name_template: omniops/halo:{{.Tag}}
image_templates:
- omniops/halo:{{.Tag}}-amd64
- omniops/halo:{{.Tag}}-arm64
- name_template: omniops/halo:latest
image_templates:
- omniops/halo:{{.Tag}}-amd64
- omniops/halo:{{.Tag}}-arm64
```

스냅샷 빌드 설정 (.goreleaser-snapshot.yaml)

개발/테스트 목적:

- Linux/AMD64만 지원

- 바이너리 아카이브 생성 안 함
- 모든 서비스 포함 (anvilproxy 포함)
- 태그: `{{.ShortCommit}}`, `main`

Docker 컨테이너 분석

공통 Dockerfile 패턴

최소 이미지 구조:

```
FROM scratch

# Install ca-certificates (for https to rollups)
COPY --from=alpine:latest /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/

# Copy binary and rename to /app
COPY <binary> /app

# Mount config directory
VOLUME ["/<service>"]

# Set working directory
WORKDIR /<service>
```

```
ENTRYPOINT ["/app"]
```

주요 특징:

1. **최소 이미지**: `FROM scratch` 로 최소 크기 달성
2. **SSL 인증서**: HTTPS 통신을 위한 CA 인증서 포함
3. **볼륨 마운트**: 설정 파일을 위한 볼륨 정의
4. **작업 디렉토리**: 자동 설정 파일 읽기를 위한 경로 설정

Halo 컨테이너 특수 사항

```
FROM scratch


# Install ca-certificates (for https to rollups)
COPY --from=alpine:latest /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/


# Create /tmp directory (default cometBFT --temp-dir)
COPY --from=alpine:latest /tmp /tmp


COPY halo /app


HEALTHCHECK CMD ["/app", "ready"]


# Mount home directory at /halo
VOLUME ["/halo"]
```

```
ENTRYPOINT ["/app"]

CMD ["run"]
```

특징:

- **헬스체크**: `HEALTHCHECK CMD ["/app", "ready"]`
- **임시 디렉토리**: CometBFT를 위한 `/tmp` 디렉토리
- **기본 명령어**: `CMD ["run"]`

Halovisor 컨테이너 (업그레이드 지원)

```
# Docker build args

ARG OMNI_COSMOVISOR_VERSION=v0.7.0

ARG HALO_VERSION_0_GENESIS=v0.8.1

ARG HALO_VERSION_1_ULUWATU=v0.12.0

ARG HALO_VERSION_2_MAGELLAN=v0.14.0

ARG HALO_VERSION_3_DRAKE=v0.16.0

ARG HALO_VERSION_N_LATEST=main


# Build stages

FROM omniops/cosmovisor:${OMNI_COSMOVISOR_VERSION} AS build-cosmovisor

FROM omniops/halo:${HALO_VERSION_0_GENESIS} AS build-0-genesis

FROM omniops/halo:${HALO_VERSION_1_ULUWATU} AS build-1-uluwatu

FROM omniops/halo:${HALO_VERSION_2_MAGELLAN} AS build-2-magellan
```

```
FROM omniops/halo:${HALO_VERSION_3_DRAKE} AS build-3-drake

FROM omniops/halo:${HALO_VERSION_N_LATEST} AS build-4-earhart
```

멀티 스테이지 빌드:

1. **Cosmovisor**: 업그레이드 관리자
2. **다중 Halo 버전**: 각 네트워크 업그레이드 버전
3. **런타임 스테이지**: 모든 바이너리를 포함한 최종 이미지

Cosmovisor 환경 변수:

```
ENV DAEMON_HOME=/halo

ENV DAEMON_NAME=halo

ENV DAEMON_ALLOW_DOWNLOAD_BINARIES=false

ENV DAEMON_RESTART_AFTER_UPGRADE=true

ENV UNSAFE_SKIP_BACKUP=true

ENV COSMOVISOR_TIMEFORMAT_LOGS=rfc3339nano

ENV COSMOVISOR_CUSTOM_ROOT=/halovisor

ENV COSMOVISOR_CUSTOM_CURRENT_LINK=/halo/halovisor-current

ENV COSMOVISOR_CUSTOM_CURRENT_UPGRADEINFO=/halo/halovisor-upgradeinfo.json
```

빌드 스크립트 분석

단일 Docker 빌드 (scripts/build_docker.sh)

```
#!/usr/bin/env bash
```

```
APP="${1}"
PARENT_DIR="${2}"  # Defaults to repo root
GITREF="${3}"  # Defaults to git rev-parse HEAD
ARC="${4}"  # Defaults to go env GOARCH

# GoReleaser로 단일 타겟 빌드
GOOS=linux  GOARCH="${ARC}"  goreleaser  build  --single-target  --snapshot  --c

cd  dist/${APP}* || exit  1

# Docker 이미지 빌드
docker  build  -f  "../../${PARENT_DIR}/${APP}/Dockerfile"  .  -t  "omniops/${AP
```

기능:

1. **단일 앱 빌드**: 특정 서비스만 빌드
2. **아키텍처 지정**: 로컬 또는 지정된 아키텍처
3. **Git 참조 태깅**: 커밋 해시 기반 태그

Halovisor 빌드 (scripts/halovisor/build.sh)

```
#!/usr/bin/env bash

set  -e
```

```
SCRIPT_DIR=$( cd -- "$( dirname -- "${BASH_SOURCE[0]}" )" &> /dev/null && pwd

HALO_VERSION_N_LATEST=$(git rev-parse --short=7 HEAD)
echo "Using head as HALO_VERSION_N_LATEST: ${HALO_VERSION_N_LATEST}"

IMAGEREF="omniops/halovisor:${HALO_VERSION_N_LATEST}"
IMAGEMAIN="omniops/halovisor:main"

docker build \
--build-arg HALO_VERSION_N_LATEST="${HALO_VERSION_N_LATEST}" \
-t "${IMAGEREF}" \
-t "${IMAGEMAIN}" \
"${SCRIPT_DIR}"
```

버전 관리 시스템

Git 태그 기반 버전 관리 (scripts/go_tag.sh)

```
#!/bin/bash

N=$1
count=0
for tag in $(git tag --sort=-version:refname); do # Sort by *semver*
```

```
# Return Nth match
if [[ $count -eq $N ]];  then
echo  "$tag"
exit  0
fi

((count++))
done
```

기능:

- **시맨틱 버전 정렬**: `--sort=-version:refname`
- **N번째 태그 추출**: 현재(0), 이전(1) 태그 가져오기
- **JS SDK 태그 필터링**: Go 태그만 추출

Foundry 버전 관리

```
package scripts

import (
_  "embed"
)

//go:embed foundry_version.txt
var  foundryVersion  string
```

```go
func FoundryVersion() string {
return foundryVersion
}
```

특징:

- **임베드된 버전**: `foundry_version.txt` 파일 내용 임베드
- **빌드 시 주입**: Docker 빌드 시 환경 변수로 전달

배포 자동화 플로우

개발 환경 배포

1. **로컬 빌드**:

```
make build-docker
```

2. **Devnet 배포**:

```
make devnet-deploy MANIFEST=devnet1
```

3. **테스트 실행**:

```
make e2e-run MANIFEST=devnet1
```

프로덕션 배포

1. **태그 생성**: Git 태그 푸시
2. **GoReleaser 실행**: GitHub Actions에서 자동 실행
3. **Docker 이미지 빌드**: 멀티 아키텍처 이미지 생성
4. **레지스트리 푸시**: `omniops/*` 네임스페이스로 푸시
5. **매니페스트 생성**: 멀티 아키텍처 매니페스트 생성

최적화 및 보안 고려사항

이미지 크기 최적화

1. **Scratch 베이스**: 최소 이미지 크기
2. **정적 링크**: `CGO_ENABLED=0` 로 정적 바이너리 생성
3. **스트립 바이너리**: `-s -w` 링커 플래그로 디버그 정보 제거

보안 강화

1. **CA 인증서**: HTTPS 통신을 위한 인증서 포함
2. **비루트 실행**: 별도 사용자 없이 안전한 실행
3. **읽기 전용 파일시스템**: 볼륨 마운트를 통한 설정 관리

운영 편의성

1. **헬스체크**: 컨테이너 상태 모니터링
2. **볼륨 마운트**: 설정 파일 외부 관리
3. **환경 변수**: 런타임 설정 조정
4. **로그 형식**: 구조화된 로그 출력

결론

Omni의 Docker 빌드 및 배포 시스템은 다음과 같은 특징을 가집니다:

1. **자동화**: GoReleaser를 통한 완전 자동화된 빌드
2. **멀티 플랫폼**: AMD64, ARM64 아키텍처 지원
3. **최적화**: 최소 이미지 크기와 보안 강화
4. **업그레이드 지원**: Halovisor를 통한 무중단 업그레이드
5. **개발 친화적**: 로컬 개발과 테스트를 위한 도구 제공

이러한 시스템을 통해 Omni는 안정적이고 확장 가능한 배포 파이프라인을 구축하고 있습니다.